

Ex.No : 9 USE GHIDRA TO DISASSEMBLE AND ANALYSE THE MALWARE CODE
Date :

Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
 - To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
 - To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.
-

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- Overview: Brief description of Ghidra, its installation, and how it is used for malware analysis.
- Requirements: List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- Project Outline: A breakdown of the tutorial steps, linking to various sections or files within the repository.

2. Tutorial Steps (Markdown or Jupyter Notebooks)

- Step-by-step instructions, broken down into digestible sections, such as:
 - Environment Setup: Instructions on setting up a virtualized and isolated environment.
 - Initial Analysis: How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - Function Analysis: Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - String and Import Analysis: How to locate and interpret strings and imports related to malicious behavior.
 - Advanced Techniques: Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3. Example Scripts (Python or Java)

- Scripts to automate repetitive analysis tasks, such as:
 - Extracting and listing all string references.
 - Identifying network-related functions or common indicators of malware.

- Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).
- Script Examples: Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).

4. Sample Data (Benign Binary or Hex Dump)

- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
- Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
- Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.

5. Documentation and Comments

- Include explanations of the code within each script, detailing what each function does.
- Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.

6. Report Template

- A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:
 - Summary of Analysis: Brief description of the suspected malware.
 - Function Analysis: Summary of the key functions, their purpose, and the findings.
 - Behavioral Indicators: Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
 - Recommendations: Suggestions on further actions, such as containment, eradication, or further analysis.

Example Repository Structure

plaintext

ghidra-malware-analysis-project/

```

|— README.md
|— docs/
|   |— Tutorial_Step1.md
|   |— Tutorial_Step2.md
|   |— Tutorial_StepN.md

```

```

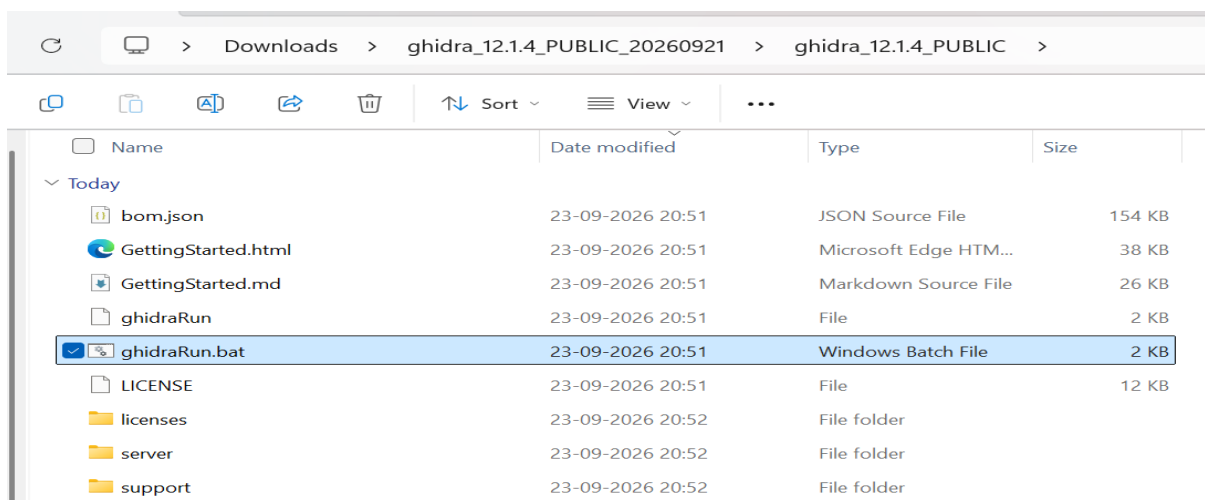
├── scripts/
│   ├── extract_strings.py
│   ├── network_analysis.py
│   └── label_functions.py
├── samples/
│   ├── benign_sample1.bin
│   └── benign_sample2.hex
├── templates/
│   └── analysis_report_template.md
└── screenshots/
    ├── function_graph_example.png
    └── decompiler_example.png

```

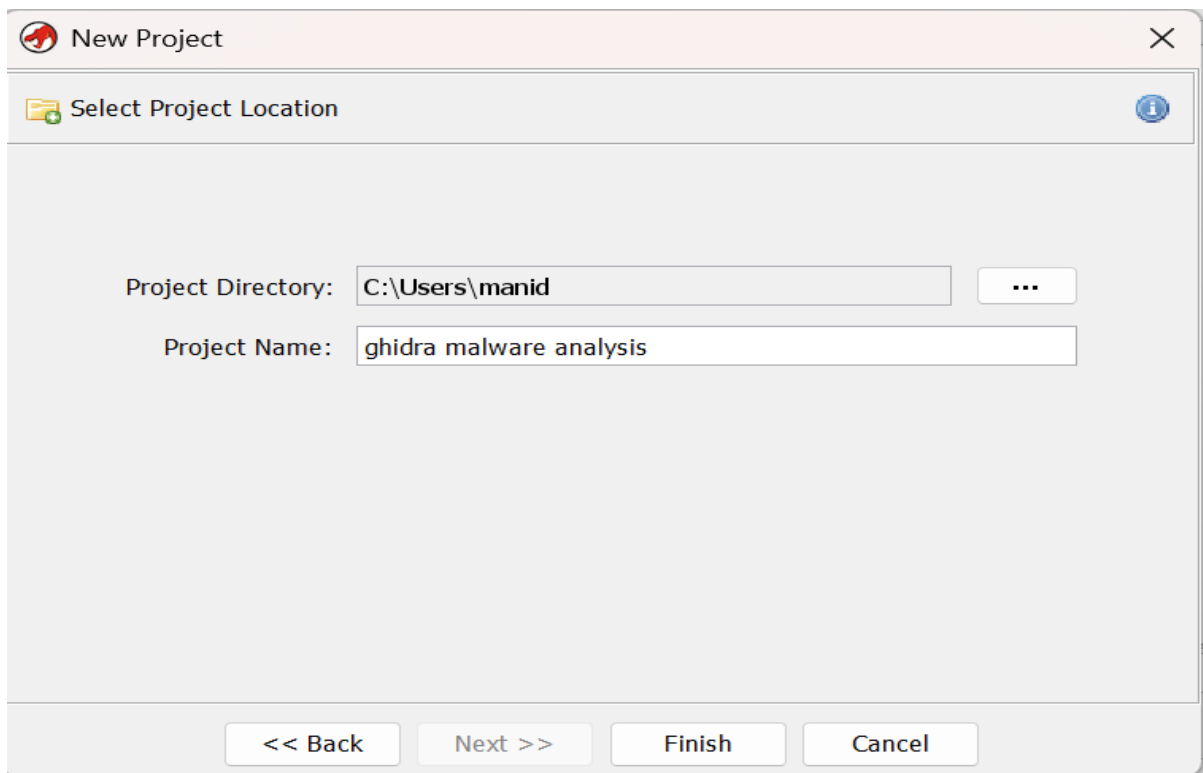
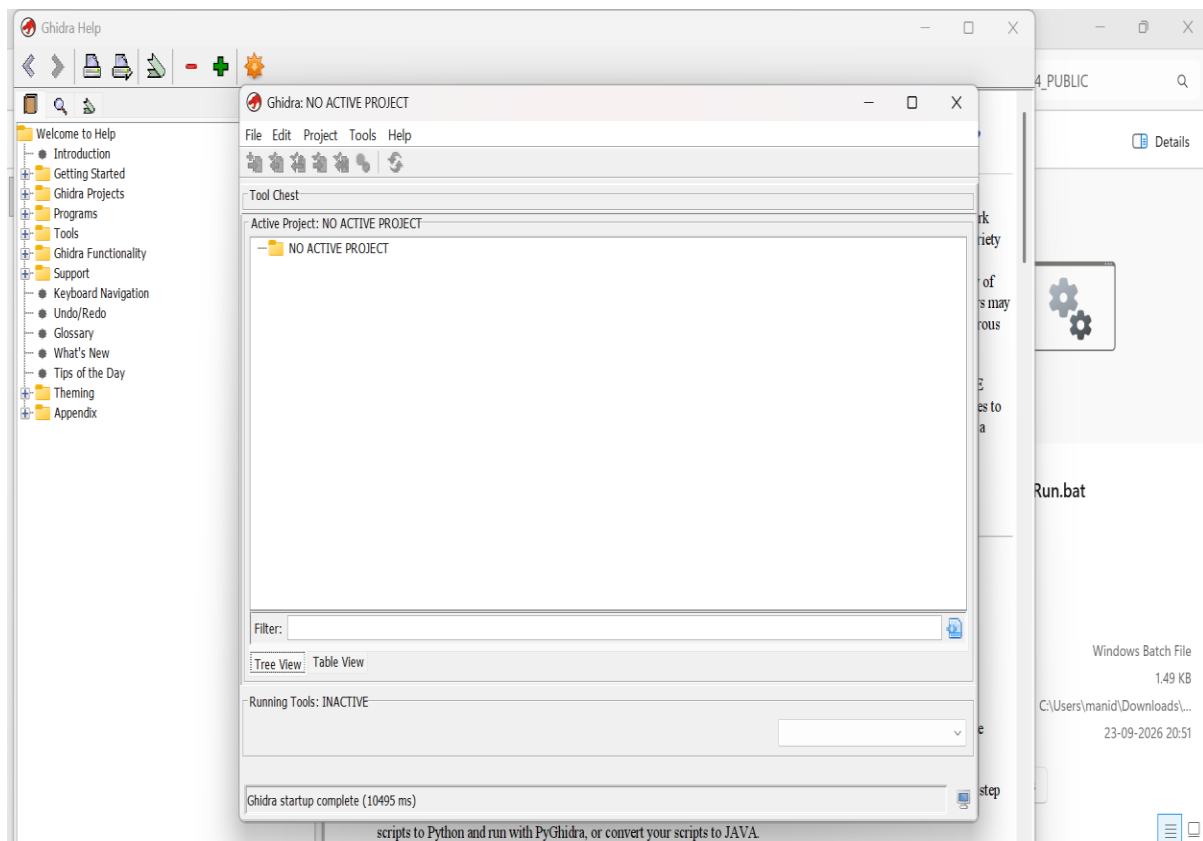
Project Development and Future Work

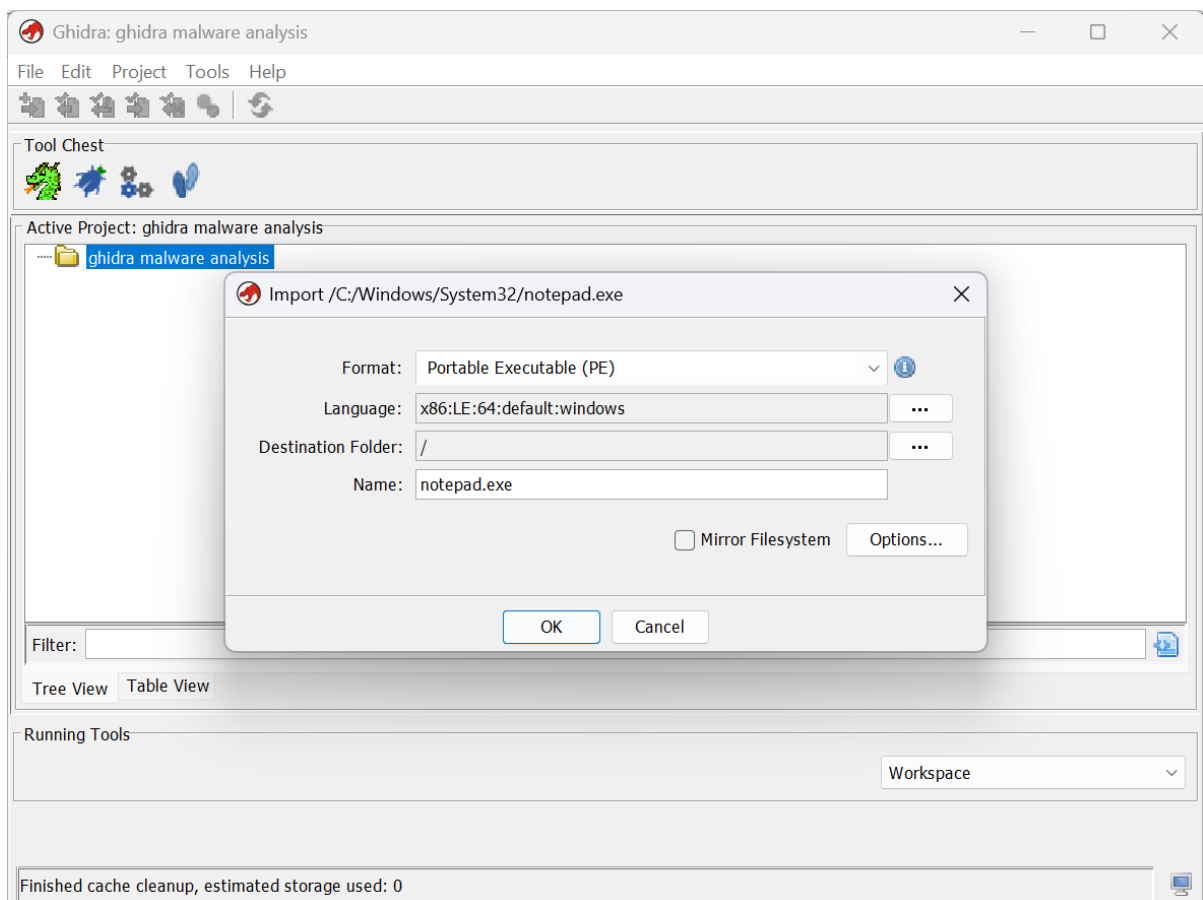
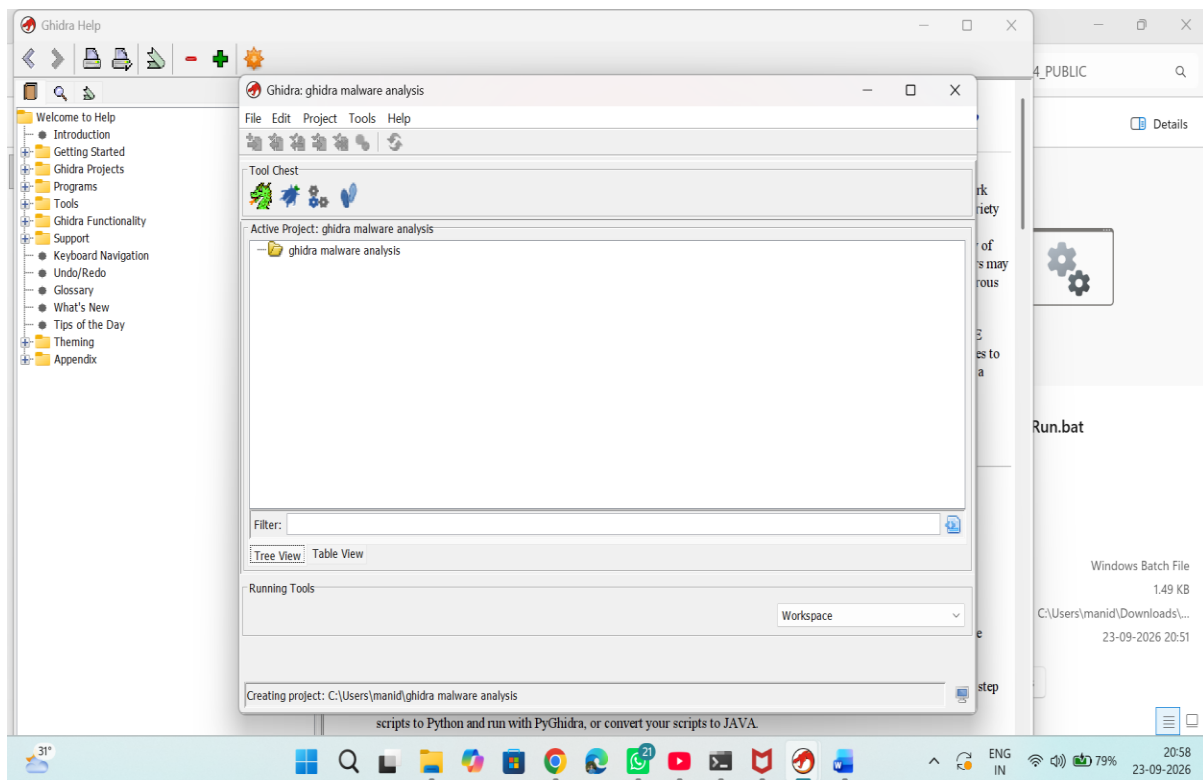
- **Expand Scripts:** Continue adding Ghidra scripts to automate complex analysis tasks.
- **Community Contributions:** Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- **Advanced Topics:** Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

SCREENSHOTS :



Name	Date modified	Type	Size
▼ Today			
bom.json	23-09-2026 20:51	JSON Source File	154 KB
GettingStarted.html	23-09-2026 20:51	Microsoft Edge HTML Document	38 KB
GettingStarted.md	23-09-2026 20:51	Markdown Source File	26 KB
ghidraRun	23-09-2026 20:51	File	2 KB
ghidraRun.bat	23-09-2026 20:51	Windows Batch File	2 KB
LICENSE	23-09-2026 20:51	File	12 KB
licenses	23-09-2026 20:52	File folder	
server	23-09-2026 20:52	File folder	
support	23-09-2026 20:52	File folder	





Import Results Summary

Project File Name: notepad.exe
Last Modified: Wed Sep 23 21:06:30 IST 2026
Readonly: false
Program Name: notepad.exe
Language ID: x86:LE:64:default (4.9)
Compiler ID: windows
Processor: x86
Endian: Little
Address Size: 64
Minimum Address: 140000000
Maximum Address: 140059fff
of Bytes: 366400
of Memory Blocks: 9
of Instructions: 0
of Defined Data: 2846
of Functions: 453
of Symbols: 912
of Data Types: 87
of Data Type Categories: 5
Compiler: visualstudio:unknown
Created With Ghidra Version: 12.1.4
Date Created: Wed Sep 23 21:06:13 IST 2026
Executable Format: Portable Executable (PE)
Executable Location: /C:/Windows/System32/notepad.exe
Executable MD5: 8a1d8175ccca97054cdb25acbb4cc07e
Executable SHA256: 468ffe129c395abf6b21a09efdf261910a95fb98aa982ead73caa7b2b684577e
FSRL: file:///C:/Windows/System32/notepad.exe?MD5=8a1d8175ccca97054cdb25

Additional Information
Loading file:///C:/Windows/System32/notepad.exe?MD5=8a1d8175ccca97054cdb25acbb4cc07e...

Welcome to Help | File | Edit | Project | Tools | Help

eBrowser: ghidra malware analysis:/notepad.exe

t Analysis BSim Graph Navigation Search Select Tools Window Help

Trees | Listing: notepad.exe

notepad.exe
Headers
.text
fothk
.rdata
.data
.pdata
didat

n Tree X

ol Tree X

Imports
Exports
Functions
Labels
Classes
Namespaces

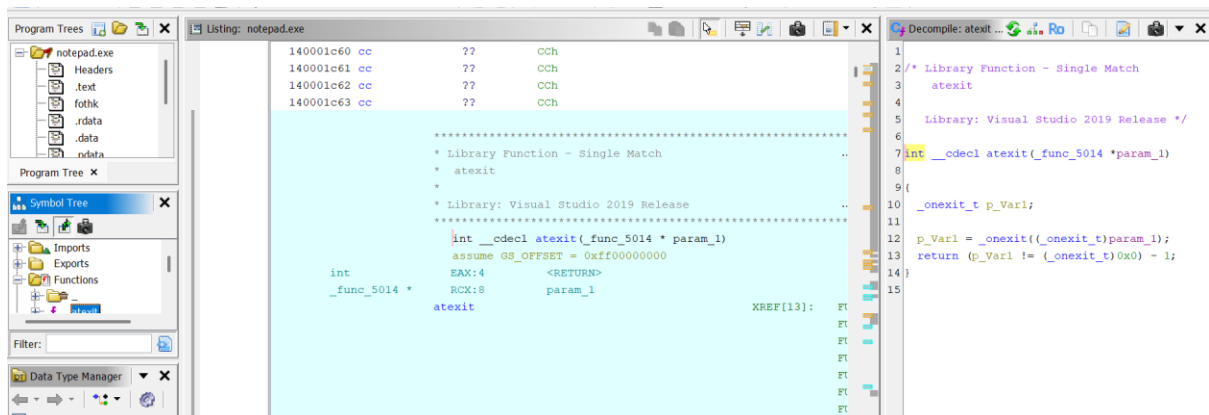
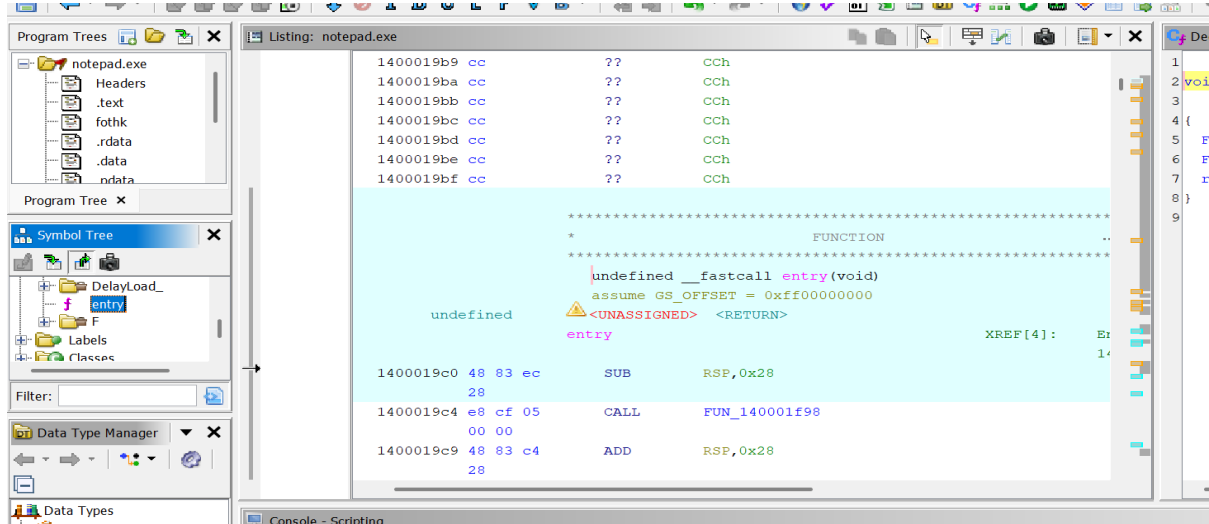
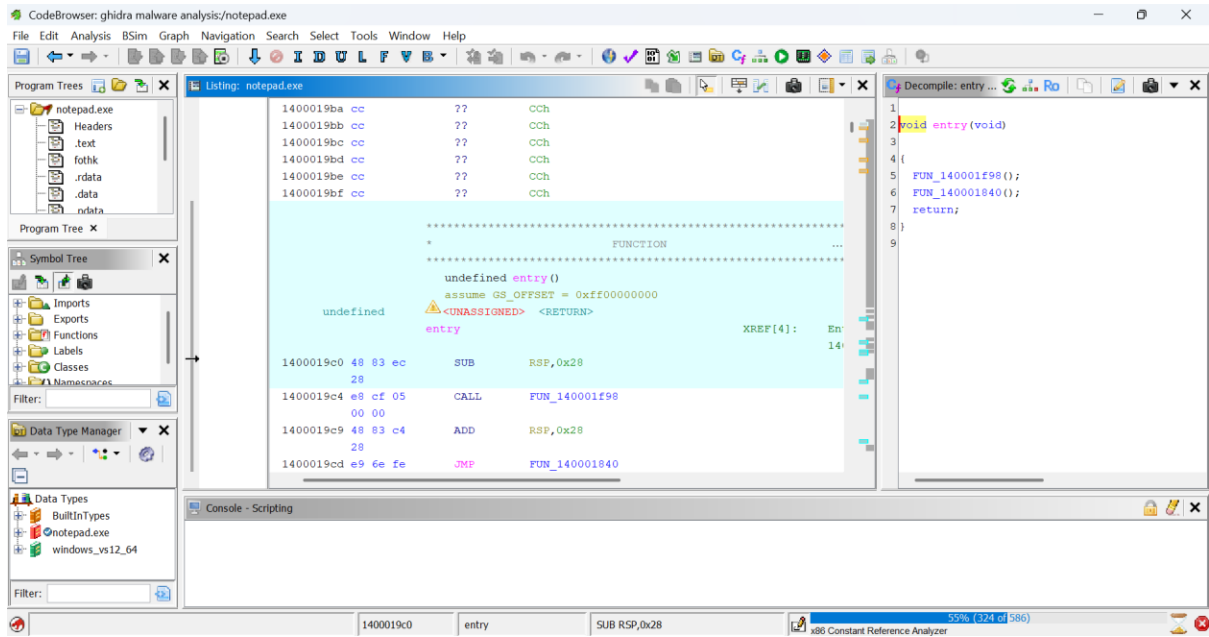
Type Manager X

Analyze?

notepad.exe has not been analyzed. Would you like to analyze it now?

Yes No No (Don't ask again)

```
//  
// Headers  
// ram:140000000-ram:140000fff  
//  
  
assume DF = 0x0 (Default)  
IMAGE_DOS_HEADER_140000000 XREF[1]:  
140000000 4d 5a 90 IMAGE_DO...  
00 03 00  
00 00 04 ...  
140000080 30 76 2d IMAGE_PT...  
f7 7  
53 a  
1400000f0 00  
1400000f1 00  
1400000f2 00  
1400000f3 00  
1400000f4 00  
1400000f5 00 ?? 00h  
1400000f6 00 ?? 00h  
1400000f7 00 ?? 00h  
1400000f8 50 45 00 IMAGE_NT...  
00 64 86  
08 00 04 ...
```



String Search [CodeBrowser: ghidra malware analysis:/notepad.exe]

Edit Help

String Search - 737 items - [notepad.exe, Minimum size = 5, Align = 1]

...	Loca...	Label	Code Unit	String View	Strin...	Le...	Is Word
	14000004d		db 21h (byte[64]e_progra...	"!This program cannot be run in DO...	string	45	true
	140000200	IMAGE_SEC...	IMAGE_SECTION_HEADER	".text"	string	6	false
	140000227		SectionFlags IMAGE_SCN_C...	".fothk"	string	7	false
	14000024f	IMAGE_SEC...	SectionFlags IMAGE_SCN_C...	".rdata"	string	8	true
	140000277		SectionFlags IMAGE_SCN_C...	".data"	string	7	false
	1400002a0		IMAGE_SECTION_HEADER	".pdata"	string	7	true
	1400002c7		SectionFlags IMAGE_SCN_C...	".@.didat"	string	8	true
	1400002f0		IMAGE_SECTION_HEADER	".rsrc"	string	6	false
	140000317		SectionFlags IMAGE_SCN_C...	".@.reloc"	string	8	true
	14002a7c0	s_Unknown_...	ds "Unknown exception"	"Unknown exception"	string	18	true
	14002a7d8	s_bad_alloc...	ds "bad allocation"	"bad allocation"	string	15	true
	14002a7e8	s_bad_array...	ds "bad array new length"	"bad array new length"	string	21	true
	14002a800	s_ADVAPI32...	ds "ADVAPI32.dll"	"ADVAPI32.dll"	string	13	true
	14002a810	s_COMDLG3...	ds "COMDLG32.dll"	"COMDLG32.dll"	string	13	true
	14002a820	s_PROPSYS...	ds "PROPSYS.dll"	"PROPSYS.dll"	string	12	true
	14002a830	s_SHELL32...	ds "SHELL32.dll"	"SHELL32.dll"	string	12	true
	14002a890	s_WINSPOOL...	ds "WINSPOOL.DRV"	"WINSPOOL.DRV"	string	13	true
	14002a8a0	s_urlmon.dl...	ds "urlmon.dll"	"urlmon.dll"	string	11	true
	14002a8b0	s_Exception...	ds "Exception"	"Exception"	string	10	true
	14002a8c0	s_ReturnNt...	ds "ReturnNt"	"ReturnNt"	string	9	true
	14002a8d0	s_ReturnHr...	ds "ReturnHr"	"ReturnHr"	string	9	true
	14002a8dc	s_LogNt_140...	ds "LogNt"	"LogNt"	string	6	false
	14002a8e4	s_LogHr_140...	ds "LogHr"	"LogHr"	string	6	false
	14002a8f0	s_FailFast 1...	ds "FailFast"	"FailFast"	string	9	true

Filter:

☐ Auto Label Offset: 0 Dec Preview:

☐ Include Alignment Nulls

☐ Truncate If Needed

Make String Make Char Array

Program Trees

Listing: notepad.exe

Program Tree X

Symbol Tree X

Imports

DecryptFileW

DuplicateEncry

API-MS-WIN-BASE

Filter:

Data Type Manager

```

140027567 cc    ??    CCh
140027568 cc    ??    CCh
140027569 cc    ??    CCh
14002756a cc    ??    CCh
14002756b cc    ??    CCh
14002756c cc    ??    CCh
14002756d cc    ??    CCh

*****
*                               FUNCTION
*****

undefined * __fastcall Catch_All@14002756e(void)
        assume GS_OFFSET = 0xffff00000000

undefined *      RAX:8      <RETURN>
undefined8      Stack[0x10]:8 local_res10      XREF[
Catch_All@14002756e      XREF[2]: 14

14002756e 48 89 54      MOV     qword ptr [RSP + local_res10],RDX
                24 10

140027573 55          PUSH    RBP
140027574 48 83 ec      SUB     RSP,0x20
                20

140027578 48 8b ea      MOV     RBP,RDX

```

Decompile: Catch...

```

1
2 undefined * Catch_All@14002756e(void)
3
4 {
5     return &DAT_1400010c4;
6 }
7

```